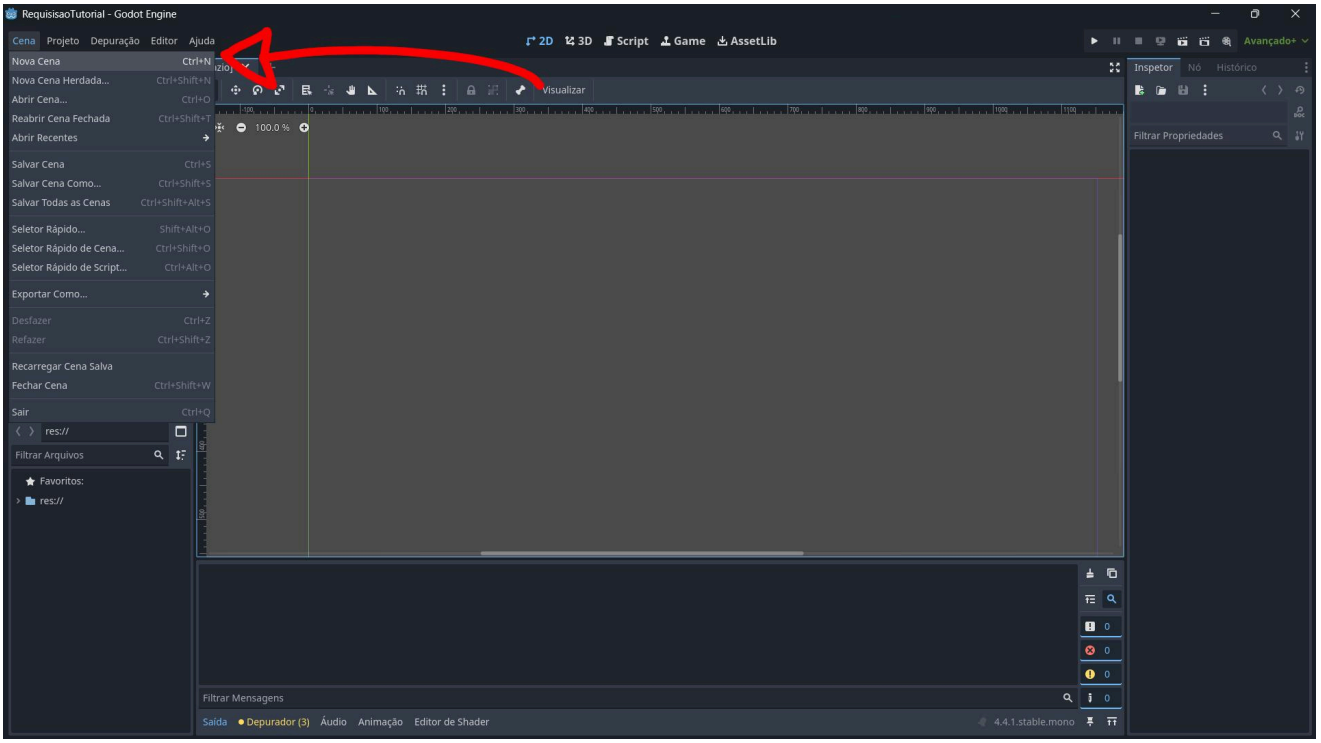
Enviar dados do jogo para um servidor, por meio de HTTPRequest no Godot

---

Parte 1: Criando Node para ser quem vai enviar dados ao servidor

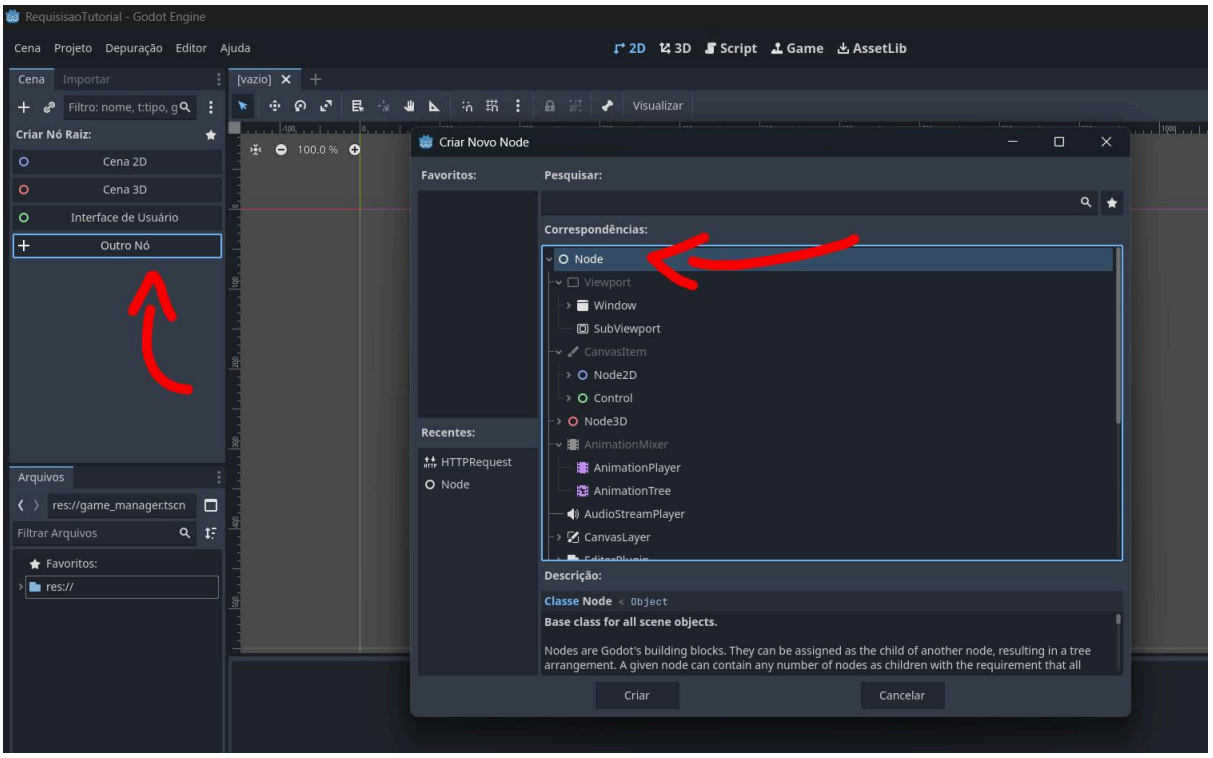
Passo 1: Crie uma nova cena

(Opcional, se quiser é só passar para o próximo passo de criar um Node, mas criando uma nova cena aumenta organização)



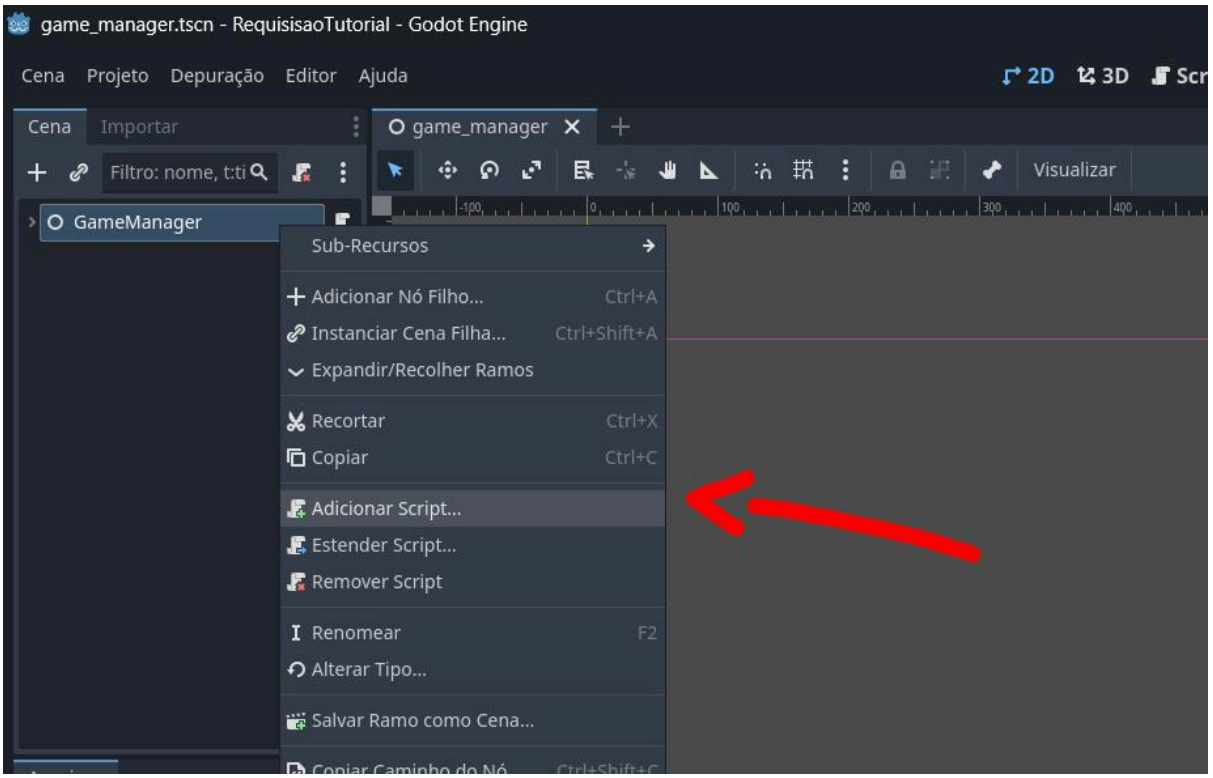
Passo 2: Crie um nó “comum”

1. Clique em “Outro Nó”
2. Selecione o nó “Node”, o ícone dele é um círculo branco



Passo 3: Crie um script para o Node

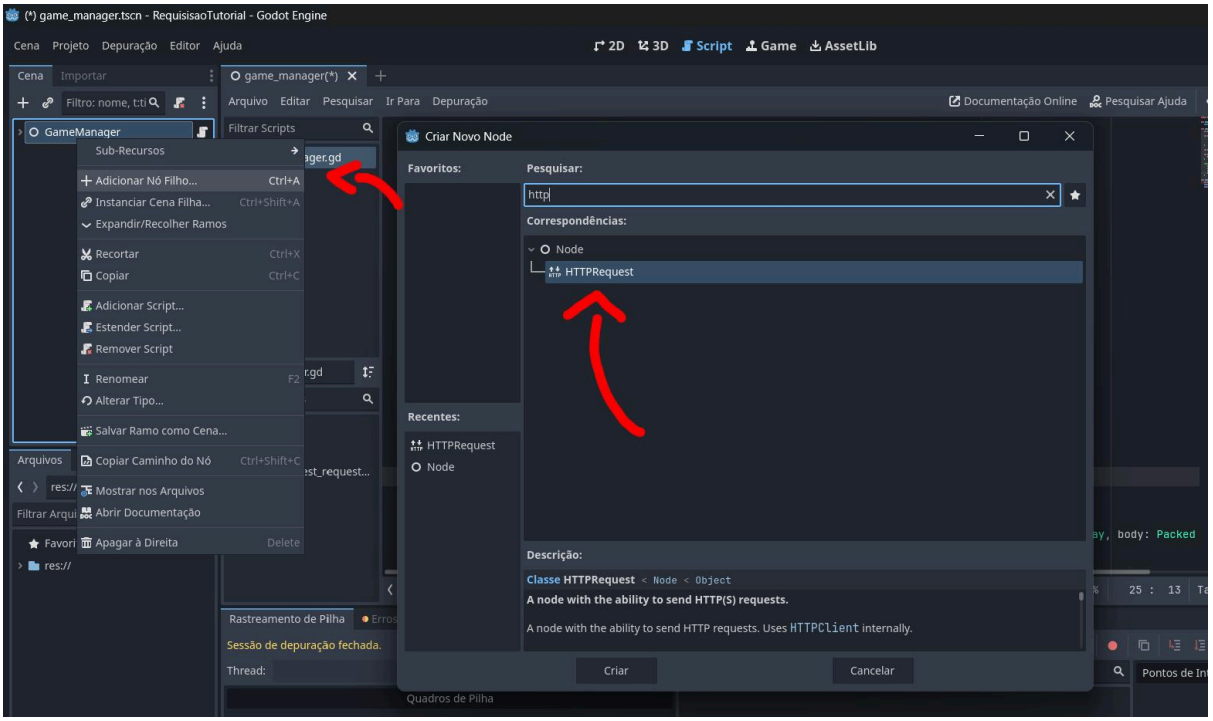
1. Clique com o botão direito no Node criado (nomeei para GameManager)
2. Selecione para “Adicionar Script”



Parte 2: Criando a requisição HTTP

Passo 1: Adicionar o node HTTPRequest

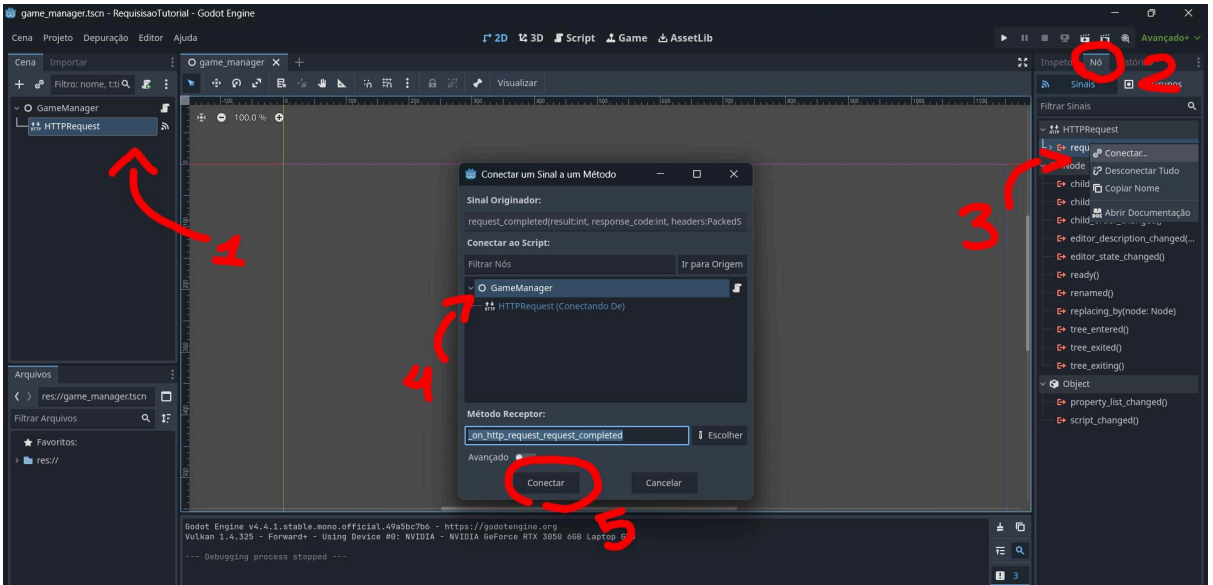
1. Clique com o botão direito no Node criado
2. Selecione “Adicionar Nó Filho”
3. Procure e clique em “HTTPRequest” para adicionar o Node



Passo 2: Conectar sinal de resposta da requisição

(Opcional, mas extremamente útil para receber feedback no Godot de quando a requisição de envio de dados dá certo)

1. Selecione o Node HTTPRequest
2. Na direita vá em “Nó” e clique com o botão direito em “request_completed”
3. Clique em “Conectar” e conecte ao primeiro Node que foi feito



Parte 3: Enviando dados para o servidor

Passo 1: Programar a conexão ao servidor

(Vou criar um código como exemplo, fique a vontade para copiar e/ou usar para estudar)

1. No primeiro Node criado é possível ver um ícone de um “pergaminho”, que quando clicado abre a tela de código do Godot, ou clique em “Script” que está ao lado do “3D” na parte superior central
2. Agora só criar o código que conecte com o servidor e manda dados, aqui meu exemplo:

```
extends Node

# Pegar o node de HTTPRequest que adicionamos
@onready var http: HTTPRequest = $HTTPRequest

# API Key usada para autenticar as requisições no servidor
var API_KEY := "SUA_API_KEY"

# Endereço base do servidor (API)
var BASE_URL := "http://ENDERECO:3005"

# Sinal emitido quando uma requisição termina
signal request_finished(type, success, data)

# Salva qual tipo de dado foi enviado (score ou gametime)
var current_request_type := ""


func _ready():
	# Conecta o sinal de resposta do HTTPRequest para tratar quando o servidor responder
	http.request_completed.connect(_on_http_request_request_completed)

# Envia apenas o Score do jogador
func enviar_score(score):
	# Monta a URL completa do endpoint
	var endpoint = BASE_URL + "/v1/scores"

	# Dados que serão enviados para o servidor
	var dados = {
		"score": score
	}

	# Chama a função genérica de envio
	enviar_request(endpoint, dados, "score")

# Envia apenas o tempo de jogo do jogador
func enviar_gametime(gametime):
	# Monta a URL completa do endpoint
	var endpoint = BASE_URL + "/v1/gametime"
```

```
	# Dados que serão enviados para o servidor
	var dados = {
		"gametime": gametime
	}

	# Chama a função genérica de envio
	enviar_request(endpoint, dados, "gametime")


# Função responsável por montar e enviar a requisição HTTP
func enviar_request(url, dados: Dictionary, type: String):
	# Converte os dados para formato JSON
	var json = JSON.stringify(dados)

	# Headers da requisição
	# Além do JSON, também envia a API key para autenticação
	var headers = [
		"Content-Type: application/json",
		"x-api-key: " + API_KEY
	]

	# Guarda o tipo da requisição (score ou gametime)
	current_request_type = type

	# Executa a requisição HTTP (POST)
	var error = http.request(url, headers, HTTPClient.METHOD_POST, json)
```

Passo 2: Lembre se de colocar o IP do servidor e a API KEY

A Fim de conectar corretamente com o servidor é importante preencher *var API_KEY* e *var BASE_URL*:

```
# API Key usada para autenticar as requisições no servidor
var API_KEY := "SUA_API_KEY"

# Endereço base do servidor (API)
var BASE_URL := "http://ENDERECO:3005"
```

Parte 4: Respostas da API

É importante saber as respostas que o servidor pode mandar, que são as seguintes:

- 200 - Sucesso
- 401 - API Key inválida
- 422 - Dados inválidos
- 429 - Muitas requisições

Parte 5: Como usar

Aqui vou mostrar algumas maneiras de usar o Node que criamos para enviar os dados

Passo 1: Exemplo simples

(Opcional seguir aqui, mas é um exemplo de como funciona as funções do código)
Para usar as funções basta chamar o Node criado no script que você quiser, e fazer elas receberem o valor que você quiser, eu fiz outros dois scripts globais para acompanhar os pontos (*score*) e o tempo (*gametime*) do jogador, aqui uma função exemplo:

```
func send_data():
	# Aqui eu uso o ScoreManager.score que é um script global de pontuação
	var score = ScoreManager.score

	# Assim como o ScoreManager, o GameTimer.time também é um script global
	var time = GameTimer.time

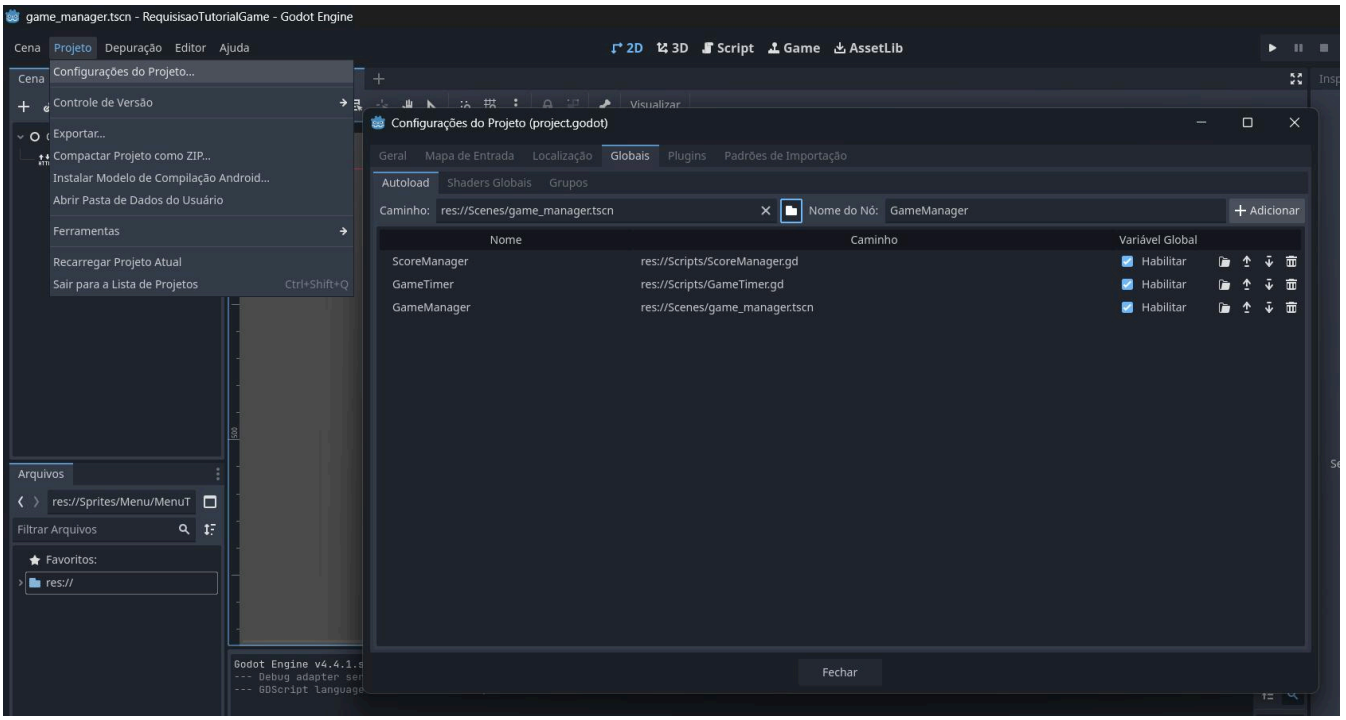
	# Agora só enviar chamando o Node de requisição que criamos
	# OBS: é importante saber que não é bom colocar duas funções de envio
	# uma logo abaixo da outra, pois ele tenta enviar as duas ao mesmo tempo
	# e acaba mandando apenas uma (a primeira)

	GameManager.enviar_score(score)
	GameManager.enviar_gametime(time)
```

Passo 2: Como tornar o Node em global

(*também opcional, mas facilita bastante o uso*)
Você pode chamar o Node da maneira que preferir, mas colocá-lo como global facilita em chamá-lo para qualquer script sem precisar carregar ele antes:

1. Na aba “Projeto” no canto superior esquerdo clique em “Configurações do Projeto…”
2. Depois vá em “Globais” e no ícone da pasta selecione o arquivo da cena que criamos que possui o Node de controle de requisições
3. Então só escolher um nome e clicar em “Adicionar”



Passo 3: Respostas do servidor

Para acompanhar a resposta do servidor podemos usar a função que foi criada no passo 2 da parte 2 desse documento, aqui um exemplo de código:

```
# Função chamada automaticamente quando o servidor responde
func _on_http_request_request_completed(result: int, response_code: int, headers:
PackedStringArray, body: PackedByteArray) -> void:
	# Converte o corpo da resposta (bytes) para texto
		var resposta_texto = body.get_string_from_utf8()

		var success := false
		var data := {}

	# Para o código que o servidor respondeu ir ao console
	print(response_code)

	# Se o código HTTP for 200, deu certo
	if response_code == 200:
		success = true

		# Tenta converter a resposta JSON
		var json = JSON.parse_string(resposta_texto)
		if json:
			data = json

	# Emite um sinal avisando outras partes do jogo (como o menu) sobre o resultado da
requisição
```

```
emit_signal("request_finished", current_request_type, success, {
        "code": response_code,
        "body": data
})
```

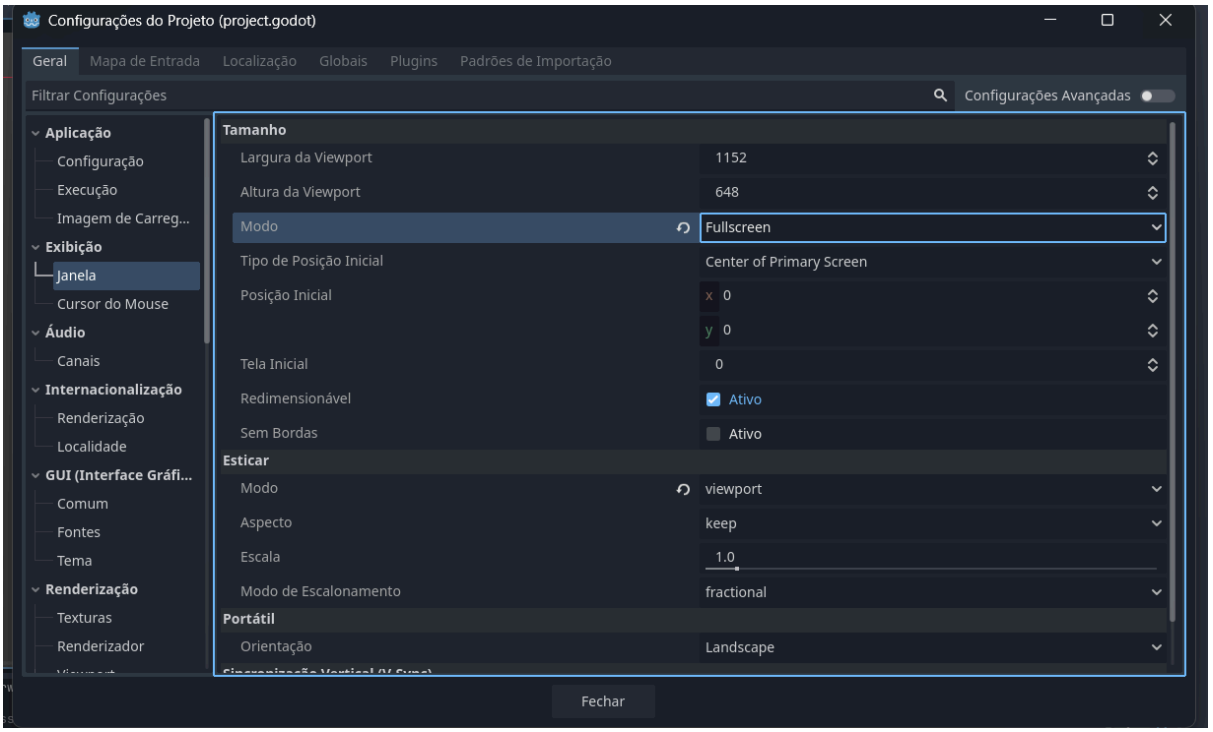
Parte 6: Enviar tempo de jogo ao saír

Uma maneira de enviar o tempo de jogo do jogador quando ele fechar o jogo

Passo 1: Colocar a tela como fullscreen

com isso o jogador não pode clicar no “X” da janela pois ela vai estar maximizada

1. Vá novamente em “Projeto” e “Configurações do Projeto…”
2. Agora em “Geral” e na aba de “Janela”
3. Mude o “Modo” do “Tamanho” de “Windowed” para “Fullscreen”
4. Mude o “Modo” do “Esticar” de “disabled” para “viewport”



Passo 2: Botão de saída

(*opcional, pois você pode fazer diferentes maneiras para sair, irei comentar sobre a ideia de um botão para isso*)
Agora o jogador precisa de outra maneira de sair, para isso você pode criar um botão que quando pressionado manda o tempo de jogo e chama “get_tree().quit()” para fechar o jogo (*no meu caso, o botão vai estar em um menu*), assim por exemplo:

```
func _on_quit_game_pressed() -> void:
	var tempo = GameTimer.time
	GameManager.enviar_gametime(tempo)
	get_tree().quit()
```

(*Nesse exemplo o "get_tree().quit()" fica logo abaixo da chamada da função de enviar o tempo, isso pode fazer com que ele não consiga enviar a tempo antes de fechar, vou exemplificar uma maneira de sair (rodar o "get_tree().quit()") apenas quando houver sucesso em enviar o dado de tempo ao servidor*)

Passo 3: Sair mandando o gametime corretamente

Ainda com o exemplo do botão, podemos remover o "get_tree().quit()" do código:

```
func _on_quit_game_pressed() -> void:
	var tempo = GameTimer.time
	GameManager.enviar_gametime(tempo)
```

E com isso podemos usar o *signal* que foi colocado no código do Request, para sinalizar que o request foi finalizado, ainda pensando que o botão está um menu, precisamos chamar esse signal para o script do menu:

```
# a função _ready(): ocorre assim que o objeto que possui a função no script entra na cena
func _ready():
	# Conecta ao signal do Node global do request para receber resposta das requisições
	GameManager.request_finished.connect(_on_request_finished)
```

Assim podemos fazer uma função que sempre ocorre quando uma requisição ocorre com sucesso, aqui por exemplo:

```
# Função chamada quando o GameManager recebe resposta do servidor
func _on_request_finished(type, success, data):
	# Se deu sucesso (HTTP 200)
	if success:
		match type:
			"score":
				# Mostra o score retornado pelo servidor no console
				print(str(data["body"]["response"]["score"]))

			"gametime":
				# Mostra o tempo retornado pelo servidor no console
				print(str(data["body"]["response"]["gametime"]))

				# Só é possível sair se enviou os dados de gametime com sucesso
				can_quit = true

# Se deu erro (401, 422, 429, etc)
	else:
		var code = data["code"]

		match type:
```

```
                "score":
                        print("Erro do score" + str(code))

                "gametime":
                        print("Erro do gametime" + str(code))
```

Com isso podemos fazer com que sempre que ocorra um erro ou um sucesso a depender do tipo de dado faça algo diferente, coloquei para que quando envie o tempo (*gametime*) com sucesso ele deixe com que a variável "can_quit" fique true, assim podemos fazer algo como isso:

```
# A função _process(delta): ocorre sempre que um frame é passado
func _process(delta):
        # Faz a verificação se o can_quit está true, se estiver fecha o jogo
        if can_quit:
                get_tree().quit()
```

Assim temos certeza que mandamos o gametime antes de fechar o jogo, as únicas maneiras agora de sair do jogo sem mandar o tempo de jogo é usando Alt + F4, desligando a máquina, no gerenciador de tarefas ou na barra de tarefas

---

E pronto!

Agora seu jogo pode se comunicar com o servidor para enviar informações de pontuação e tempo, sinta se a vontade para ver o projeto exemplo para ver como tudo foi feito.